第十五章　高效微调：LoRA 的直觉与实战

15.1　LoRA 的核心思想：冻结大矩阵，外挂小矩阵

15.1.1　一笔绕不过去的账

在上一章，我们已经用一个 0.5B 小模型跑通了一次完整的 SFT。但真实项目里的模型通常不会只有 0.5B。你很快会遇到 7B、14B，甚至更大的模型。流程还是那套流程，问题变成了另一个：机器扛不扛得住。

0.5B 模型做全量微调，8GB 显存还能勉强塞下。换成 7B 模型，光模型权重用 bf16 加载进显存就大约 14GB；一旦进入训练，还要存梯度、优化器状态。几项叠起来，全量微调的静态开销很容易来到上百 GB 量级。具体怎么算，后面讲显存估算的时候会拆开；这里先记住这个感觉：模型一大，全量微调就会变成一件很贵的事。

于是问题来了：之前学过的那套 SFT 流程能不能保留，但不要把模型里的所有参数都打开来训练？

可以。这就是 LoRA 要解决的问题。

15.1.2　一个朴素的念头，和它的三位前辈

既然全量微调，也就是更新模型的每一个参数，太贵了，最直觉的想法就是：能不能只更新一小部分参数，把其余的冻住不动？

这个想法不是 LoRA 首创的。在深度学习的迁移学习时代，最常见的做法就是冻结模型靠前的层，只训练最后几层。但这个做法搬到大模型上效果不太稳定：Transformer 的每一层结构几乎一样，不像 CNN 那样有明显的“前面学通用、后面学具体”的层次分工，你很难判断该冻住哪些层、放开哪些层。

沿着这个方向，研究者们做过不少尝试，比如 Adapter、Prefix Tuning、Prompt Tuning 等等，统称为“参数高效微调”方法。这里的 Adapter 指的是一种早期参数高效微调方法；后面 PEFT 保存 LoRA 权重时也会用`adapter`这个词，那时指的是“可加载的小补丁文件”，语境不一样。它们各有各的思路，也各有各的局限：有的改变了模型结构导致推理变慢，有的占用了宝贵的上下文窗口长度。2021 年，微软的研究团队提出了 LoRA。它冻住原模型，在权重矩阵旁边外挂一对小矩阵，训练完之后可以把结果合并回原来的权重里。合并之后，模型结构和原来一模一样，推理没有任何额外开销，也不占上下文窗口。

LoRA 之所以能从众多方案中胜出，不是因为效果明显更好——这几种方法的效果差距不大。关键在于工程上的优势太明显了：合并后推理零成本、不占上下文窗口、实

现简单、PEFT 库开箱即用。在工业界，这些工程优势往往比论文里零点几个百分点的效果差异重要得多。

那 LoRA 外挂的那对“小矩阵”到底是怎么回事？为什么两个小矩阵就能替代对一个大矩阵的修改？要理解这一点，我们需要先搞明白一件事：微调的时候，模型的参数到底发生了多大的变化。

15.1.3 微调前后，参数变了多少？

全量微调做的事情，本质上就是把模型里的权重矩阵 W，从训练前的值更新到训练后的值。我们把训练前后的差值叫做 ΔW（读作“delta W”）。也就是说，微调后的权重等于原始权重加上一个变化量：

$$W_{\text{new}} = W + \Delta W$$

这个写法不只是一个数学记号，它和前面学过的梯度更新是同一件事。每训练一步，优化器都会根据梯度把参数推一点：有些数字被调大一点，有些数字被调小一点。训练跑了几千步以后，这些小改动累在一起，就形成了 ΔW。

所以你可以把 ΔW 理解成一张“修改清单”：原来的 W 已经在那里了，微调真正学到的，是“在原来的基础上，该往哪里改、改多少”。全量微调的做法，是把 W 里的每个数字都放开，让这张修改清单可以在所有位置上自由变化。

ΔW 和 W 的维度完全一样大。如果 W 是一个 4096×4096 的矩阵，ΔW 也是 4096×4096。

LoRA 论文的作者发现了一件关键的事：**虽然 ΔW 的维度和 W 一样大，但 ΔW 里面真正有用的信息非常少**。更准确一点说，ΔW 不一定严格就是低秩矩阵，但它可以被一个低秩矩阵很好地近似。

“秩”这个概念在国内高校的线性代数课上通常被讲得很抽象，一堆行列式变换之后告诉你“秩等于几”，但很少有人解释秩的直觉到底是什么。这里我用一个简单的数字例子帮你把这个概念看清楚。

假设有一个 4×4 的矩阵 A：

$$\begin{matrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 6 & 8 \\ 3 & 6 & 9 & 12 \\ 0 & 1 & 0 & 2 \end{matrix}$$

你仔细看前三行：第二行 $[2,4,6,8]$ 恰好是第一行 $[1,2,3,4]$ 乘以 2。第三行 $[3,6,9,12]$ 恰好是第一行乘以 3。也就是说，第二行和第三行没有提供任何“新信息”——它们完全可以由第一行推算出来。只有第一行和第四行是真正独立的、互相推算不出来的。你可以试试：如果第一行 $[1,2,3,4]$ 乘以 0，第一个位置能对上第四行，但第二个位置就变成了 0，不是 1；如果乘以 0.5，第二个位置能对上，但第一个位置又变成了 0.5，不是 0。没有一个统一的倍数能同时对上所有位置，所以第四行确实带着第一行给不出的新信息。

这个矩阵的秩就是 2。

“秩”的直觉就是这个意思：一个矩阵里，有多少行是真正独立的、不能被其他行凑出来的。换成几何语言，每一行都可以看成一个变化方向；真正独立的行越少，说明

真正独立的方向越少。虽然这个矩阵有 4 行、16 个数字，但它的真实信息量只相当于 2 行。

再对比另一个 4×4 矩阵 B：

$$\begin{matrix} 1 & 0 & 3 & 2 \\ 0 & 5 & 1 & 4 \\ 2 & 1 & 0 & 3 \\ 1 & 3 & 2 & 0 \end{matrix}$$

这四行里，没有任何一行能被其他几行的组合凑出来。每一行都带着独立的新信息。这个矩阵的秩是 4，也就是“满秩”——它的信息量撑满了自己的维度。

现在回到 LoRA 的语境。ΔW 是微调前后权重矩阵的变化量，它的维度可能是 4096×4096，理论上秩最大可以到 4096。但实际上，ΔW 的有效变化往往可以用个位数到几十个方向来近似。几千行变化里面，绝大部分都可以被少数几行的组合表示出来。

15.1.4 为什么 ΔW 适合低秩近似

这件事并不是巧合，背后有一个很直觉的原因。

你想，微调用的数据量有多少？做指令微调的数据，通常是几千到几万条。而模型有多少参数？几十亿个。这意味着微调数据能提供给模型的调整信号非常有限：几万条数据里包含的独立信息方向，远远撑不起几十亿参数的自由变化。

打个不严格但直观的比方：你给模型看了 1 万条客服对话来做微调。这 1 万条对话虽然各不相同，但它们传递给模型的信号，归结起来可能就集中在十几个方向上，比如“语气要更礼貌”“回答要更简洁”“遇到投诉先道歉”“不确定的事情不要乱说”。这些有限的调整方向，决定了 ΔW 里只有十几个方向上有有效的变化，其余方向上的变化接近于零。

另外，还有一个更根本的原因：预训练模型已经从海量数据里学到了非常强大的通用能力。微调不是要从头教会模型一种全新的本领，而是在已有的能力基础上做一些微小的方向性调整。调整的幅度本身就不大，需要的独立调整方向自然也不多。

这个发现的意义在于：既然 ΔW 的真实信息量远小于它的维度，我们就不需要老老实实地去存储和训练一个完整的 4096×4096 的矩阵。我们可以用一种更经济的方式来表示它。

15.1.5 一个大矩阵，拆成两个小矩阵

既然可以用低秩矩阵近似 ΔW，根据线性代数的一个基本性质，这个近似量可以被分解成两个小矩阵的乘积。

我先用一个小例子带你看看这件事是怎么运作的。

假设 ΔW 是一个 4×4 的矩阵，它的秩为 1：

$$\begin{matrix} 2 & 4 & 6 & 8 \\ 1 & 2 & 3 & 4 \\ 3 & 6 & 9 & 12 \\ 0.5 & 1 & 1.5 & 2 \end{matrix}$$

你可以验证一下：第一行是 $[2,4,6,8]$，第二行是第一行的 0.5 倍，第三行是第一行的 1.5 倍，第四行是第一行的 0.25 倍。四行都指向同一个方向，只是长度不同。秩为 1。

这个矩阵虽然有 $4\times4=16$ 个数字，但因为秩只有 1，它可以被拆成两个向量的乘积：

这里的命名顺序先说明一下：LoRA 论文把左边的矩阵叫 B，右边的矩阵叫 A，所以后面一直写成 BA。这个顺序可能和你的直觉相反，但它已经是 LoRA 里常见的约定。

一个 4×1 的列向量 $B=[2,1,3,0.5]$（竖着写），和一个 1×4 的行向量 $A=[1,2,3,4]$。

B 乘以 A，得到的恰好就是原来那个 4×4 的矩阵：

$$
\begin{aligned}
BA &= \begin{bmatrix} 2 \\ 1 \\ 3 \\ 0.5 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} \\
&= \begin{bmatrix} 2\times1 & 2\times2 & 2\times3 & 2\times4 \\ 1\times1 & 1\times2 & 1\times3 & 1\times4 \\ 3\times1 & 3\times2 & 3\times3 & 3\times4 \\ 0.5\times1 & 0.5\times2 & 0.5\times3 & 0.5\times4 \end{bmatrix} \\
&= \begin{bmatrix} 2 & 4 & 6 & 8 \\ 1 & 2 & 3 & 4 \\ 3 & 6 & 9 & 12 \\ 0.5 & 1 & 1.5 & 2 \end{bmatrix}
\end{aligned}
$$

完美还原。

现在来数参数量。原来的 ΔW 矩阵有 16 个数字。拆成 B 和 A 之后呢？B 有 4 个数字，A 有 4 个数字，一共 8 个。用 8 个参数就精确表示了原来 16 个参数的信息，因为那 16 个数字本来就只有 1 个方向上的独立信息。

这只是秩为 1 的情况。如果秩为 2 呢？你可以把它理解成两组秩为 1 的外积叠在一起：

$$\Delta W = B_1A_1 + B_2A_2$$

第一组负责一种变化方向，第二组负责另一种变化方向。比如第一组让模型学会“语气更礼貌”，第二组让模型学会“回答更简洁”。两个方向加起来，就比秩为 1 的旁路多了一种调整自由度。

从矩阵形状上看，秩为 2 时，B 就变成一个 4×2 的矩阵（8 个参数），A 变成一个 2×4 的矩阵（8 个参数），总共 16 个参数，和原来的数量一样，看起来没省。但这是因为 4×4 的矩阵本来就太小了，秩为 2 已经占到维度 4 的一半。换成真实模型里 4096×4096 的矩阵，秩为 16 只占维度的千分之四，省参效果就会变得非常惊人。

我们把维度放大，看看真实模型里的数字。

一个 7B 模型里，注意力模块的权重矩阵 W_Q，维度通常是 4096×4096，有 $4096\times4096\approx1678$ 万个参数。

如果我们决定用 $r=16$ 的低秩近似来表示 ΔW（LoRA 里 r 就是你手动设定的这个值），那就把 ΔW 拆成一个 4096×16 的矩阵 B 和一个 16×4096 的矩阵 A。参数量是多少？

B的参数量：$4096 \times 16 = 65{,}536$
A的参数量：$16 \times 4096 = 65{,}536$
合计：131,072，约 13 万。

1678 万 vs 13 万。LoRA 的参数量不到全量微调的百分之一。

图 15.1直观地展示了这个参数量的落差：大正方形是原来需要存储的 ΔW，旁边两个蓝色细长条是 LoRA 实际训练的 B 和 A。

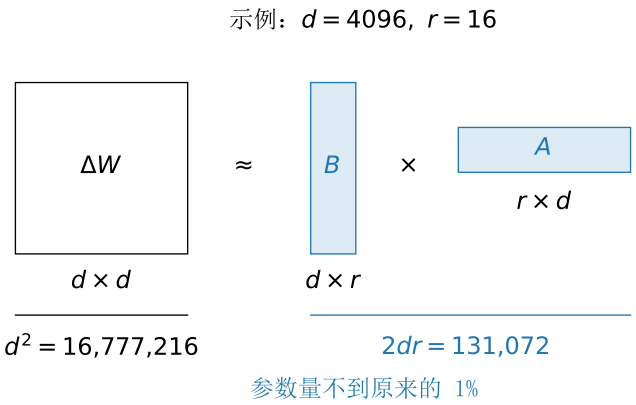


图 15.1: 低秩分解的参数量对比（示意图，矩阵比例非等比缩放）。左侧大正方形代表原始变化量 ΔW，右侧两个蓝色细长条代表分解后的 B 和 A。以 $d = 4096$、$r = 16$ 为例，参数量从 1678 万压缩到 13 万，不到原来的 1%。

这个压缩比会随着原始矩阵维度的增大变得越来越惊人——维度越大，r 相对越小，节省的比例越高。这也是为什么 LoRA 在大模型上特别有效：模型越大，参数冗余越多，低秩近似的空间越大。

15.1.6 LoRA 是怎么接进模型里的

理解了“用两个小矩阵代替一个大矩阵”的数学之后，我们来看 LoRA 在工程上到底是怎么做的。

原始模型里，某一层的计算过程是：输入 x 经过权重矩阵 W，得到输出 h。这里先按单个向量来写：x 是一个 d 维输入向量，W 是把它映射到输出空间的权重矩阵。实际训练里会有 batch 维度，但矩阵乘法的核心关系一样。

$$h = Wx$$

LoRA 的做法是：W 完全冻住，不参与训练。在 W 旁边外挂一对小矩阵 B 和 A。前向传播的时候，输入 x 同时走两条路：

第一条路：照常走 W，得到 Wx。这是原始模型的输出。

第二条路：先经过 A，再经过 B，得到 $B(Ax)$。这是 LoRA 新加的旁路。

两条路的结果加在一起：

$$h = Wx + B(Ax)$$

图 15.2展示了这个双路径结构的全貌。

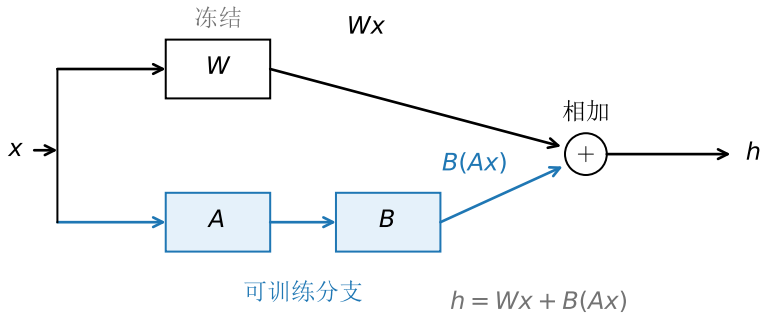


图 15.2: LoRA 的前向传播结构。输入 x 同时走两条路：上路经过冻住的原始权重 W（黑色），下路依次经过可训练的小矩阵 A 和 B（蓝色），两路结果相加得到输出 h。

训练的时候，W 的参数一个都不动，只有 A 和 B 在更新。LoRA 旁路输出的 $B(Ax)$，就是那个我们一直在讨论的“变化量” ΔW 作用在 x 上的结果。

这里有一个极其重要的细节：**A 和 B 的初始化方式是不对称的**。通常做法是：A 随机初始化，B 初始化为全零矩阵。

为什么要这样？原因很简单。LoRA 旁路的输出是 $B(Ax)$。如果 B 一开始就是零矩阵，不管 A 和 x 是什么，$B(Ax)$ 都会等于零。这样训练刚开始时，模型的输出还是原来的 Wx，不会被一条随机旁路突然搅乱。

那为什么不把 A 也初始化成零？因为旁路也要能学起来。A 随机初始化之后，Ax 通常不是零，后面的 B 虽然一开始是零，但它能在梯度更新中慢慢长出有用的修正量。你可以把它理解成：A 先提供一组可用的方向，B 从零开始决定这些方向该加多少。

如果 B 也随机初始化，训练还没开始，旁路就已经往原模型输出里加了一堆随机噪声。模型得先花力气把这堆噪声抵消掉，才能开始真正学任务。所以 LoRA 要的是“从原模型出发，慢慢加修正”，不是一上来先把原模型撞歪。

15.1.7 一个底座，多个插件

训练完 LoRA 之后，你手上有两样东西：冻住的原始权重 W，和训练好的一对小矩阵 A、B。你可以选择把 $B \times A$ 的结果加回 W，得到一个新的完整模型，这叫“合并”：

$$W_{\text{new}} = W + B \times A$$

合并之后，模型的结构和参数量与原始模型完全一样，推理的时候没有任何额外开销，速度和全量微调出来的模型没有区别。

但你也可以不合并。把 A 和 B 作为一个独立的“插件”单独保存，推理的时候按需加载。

这个“不合并”的用法打开了一个非常灵活的部署模式。

如果你玩过 AI 绘画，比如 Stable Diffusion，你可能见过这样的场景：在 Civitai 这样的社区里，有成千上万个 LoRA 文件可以下载。有的 LoRA 让画风变成二次元，有

的变成水墨画，有的变成赛博朋克。你下载一个底座模型（比如 Stable Diffusion XL），然后根据想要的风格挂上不同的 LoRA，生成出来的图片风格就跟着变。换风格不需要重新下载整个模型，只要换一个几十 MB 的 LoRA 文件就行——就像给手机换壳一样轻便。

这个思路在大语言模型的世界里完全一样。假设你的公司用一个 7B 的底座模型服务不同行业的客户：医疗问答、法律咨询、电商客服。你不需要为每个客户存一份完整的 7B 模型（每份约 14GB，十个客户就是 140GB）。你只需要存一份底座模型（14GB），加上每个客户各自的 LoRA 插件（每个几十 MB）。部署的时候，来了一个医疗客户的请求就加载医疗 LoRA，来了一个法律客户的请求就加载法律 LoRA。一个底座配十个插件，总存储不到 15GB。

这种“一个底座 + 多个可切换的插件”的部署模式，是 LoRA 在工业界大受欢迎的重要原因之一。它不仅省了训练时的显存，还省了部署时的存储和切换成本。

不过要注意两件事。第一，合并是不可逆的。一旦把 LoRA 合并进 W，你没法再把它拆出来。所以合并之前最好单独保存一份 A 和 B 的参数。第二，如果你想把两个不同的 LoRA 同时合并到一个模型里，比如想让模型既会做客服又会做翻译，简单地把两个 ΔW 加起来，效果往往不好。两个 LoRA 各自修正的方向可能互相干扰，简单叠加不等于能力叠加。如果你需要一个模型同时具备多种能力，更稳妥的做法是把多种任务的数据混在一起，训练一个 LoRA。

实战避坑 · LoRA 权重和底座模型版本不匹配的隐蔽炸弹

你训练 LoRA 用的是某个开源底座模型，过了两个月要部署的时候，同事重新从 Hugging Face 上下载了一份同名模型。名字一样，结构也一样，LoRA 能挂上去，程序不报错，模型也能正常输出，但效果莫名其妙变差了。原因很简单：LoRA 学到的是相对于特定底座权重 W 的修正量。底座换了，W 变了，LoRA 的修正方向就可能不对了。避免方法也很简单：训练 LoRA 时，把底座模型的具体版本记下来；部署时用同一份底座。

15.1.8 几个实操细节

LoRA 的核心思想到这里已经讲清楚了。下面先看两个实际使用时经常会碰到的配置细节。

缩放因子 α

前面为了讲清楚核心思想，我们一直用的是简化版公式 $h = Wx + B(Ax)$。实际使用时，LoRA 还会在旁路输出前面乘一个缩放系数。完整公式是：

$$h = Wx + (\alpha/r) \times B(Ax)$$

多了一个 α/r。r 控制的是 LoRA 旁路有多少“调整方向”，也就是容量；α/r 控制的是这些调整加到原模型输出上时有多大声。

所以调 r 的时候，最好顺手看一眼 α。如果 $r = 16, \alpha = 16$，那么 $\alpha/r = 1$；如果你把 r 改成 32，但 α 还保持 16，那么 α/r 就变成 0.5，等于把 LoRA 旁路的声音压小了一半。这样效果变了，你很难判断到底是 r 变大带来的，还是缩放变小带来的。

实操里最省心的做法是 α **通常设成和** r **相等**。比如 $r = 16$，α 就设成 16；$r = 32$，α 就设成 32。这样 α/r 始终等于 1，比较不同 r 时更直观。第一次用 LoRA，直接从这个配置开始，不用在这里反复折腾。

LoRA 旁路里的 Dropout

在 A 和 B 之间，通常还会加一个 Dropout。前面讲正则化的时候提到过，Dropout 会随机把一部分数值置零，起到防止过拟合的作用。PEFT 库里有一个`lora_dropout`参数来控制这个比例，常用值是 0.05 或 0.1。数据量少的时候适当调大一点，可以进一步防过拟合。训练结束后评测效果时，记得切回`model.eval()`；如果还停在 train 模式，LoRA 旁路里的 Dropout 仍然会随机生效，同一个输入每次结果都可能不一样。

旁路本身的几个配置讲完之后，还有一个更常见的问题：这些 LoRA 模块到底应该挂在哪些层上？

15.1.9 LoRA 加在哪些层

LoRA 挂在哪些权重矩阵上，由 PEFT 库里的`target_modules`控制。

Transformer 每一层里有不少线性矩阵：注意力模块的 W_Q、W_K、W_V、W_O，前馈网络的上投影矩阵（把维度从 d 升到更大的中间维度）和下投影矩阵（再压回 d）。LoRA 都可以挂在这些线性层旁边。

LoRA 原论文的实验默认只给 W_Q 和 W_V 加了 LoRA，很多早期教程也照搬了这个配置。但后来大量实践表明，给所有线性层都加 LoRA，效果往往更好。后面实战的时候，PEFT 库里有一个很方便的写法：把`target_modules`设成`all-linear`，它就会自动给所有线性层加上 LoRA。

常规 LoRA 主要加在线性层上。模型里还有一些其他类型的参数，比如 LayerNorm 的缩放参数、Embedding 层的参数，常规做法一般不把它们列为 LoRA 的目标模块。你在配置的时候不需要特别处理，PEFT 库的`all-linear`会按线性层去匹配目标模块，其他类型通常会被跳过。

目标模块的选择可以当成一个超参数来看。第一次做 LoRA，直接从`all-linear`开始通常最省心；如果你后面想比较显存、速度或者效果，再去试只挂 W_Q、W_V，或者只挂注意力层。

最后有一个很隐蔽的坑：如果你在`target_modules`里写了一个不存在的模块名（比如拼写错误），某些版本的 PEFT 库不会报错，只是默默跳过，LoRA 根本没加上去，你却毫不知情。配置完之后，一定要打印模型的可训练参数量来确认。如果数字是零或者远小于预期，说明配置有问题。

15.1.10 LoRA 微调到底能省多少显存

前面算过全量微调 7B 模型的静态参数相关开销大约在 100GB 以上，那 LoRA 微调呢？

冻住的模型参数仍然需要加载到显存里：模型推理要用，前向传播要用，反向传播计算梯度的时候也要用（虽然不更新，但要参与计算）。用 bf16 精度加载，这部分是 14GB。

LoRA 新增的可训练参数只有几百万个，对应的梯度和 Adam 优化器状态加起来也就几十 MB，可以忽略不计。

主要的额外开销是前向传播和反向传播过程中产生的中间激活值。这部分取决于你的 batch size 和序列长度，通常占几个 GB。LoRA 可训练参数少，不代表激活值也会跟着大幅减少。冻住的底座模型仍然要完整跑前向传播，反向传播也要经过这些层把梯度传到 LoRA 旁路，所以激活值主要看模型结构、batch size 和序列长度。

算下来，LoRA 微调一个 7B 模型（bf16 精度），大约需要 16 到 20GB 显存。24GB 的 RTX 4090 能装下。

如果配合量化技术（把冻住的模型参数从 16 位压缩到 4 位），冻住的参数从 14GB 压缩到大约 3.5GB，总显存可以压到 10GB 左右。这种“量化 +LoRA”的组合叫 QLoRA，是目前在消费级显卡上微调大模型最常用的方案。量化的具体原理后面会专门讲，这里你只需要知道有这个操作就行。

和全量微调放在一起对比：

显存构成	全量微调（7B, bf16）	LoRA 微调（7B, bf16）
模型参数	14GB（参与训练）	14GB（冻住，仅参与计算）
LoRA 新增参数	–	约几十 MB，可忽略
梯度、优化器状态和可能的 fp32 权重副本	约 90GB+（所有参数）	约几十 MB（只有 LoRA 参数）
中间激活值	数 GB	数 GB（基本相同）
总计	约 100GB+	约 16 到 20GB

省下来的主要是所有参数对应的梯度、优化器状态和可能的 fp32 权重副本这些训练态开销。全量微调时，这些开销非常大，而 LoRA 微调时它们几乎可以忽略。面试里如果让你估算 7B 模型做 LoRA 微调需要多少显存，考的也就是这套逐项拆解：冻住的模型参数约 14GB，LoRA 参数、梯度和优化器状态可以近似忽略，再加上由 batch size 和序列长度决定的激活值，总数通常落在 16 到 20GB 这个量级。

15.1.11 LoRA 微调 vs 全量微调：全景对比

把到目前为止讲的各个方面汇总成一张表，让你一眼看到两种方式的差异：

维度	全量微调	LoRA 微调
参数更新	更新全部参数	只更新外挂小矩阵
训练显存	~100GB+	~16-20GB（bf16），~10GB（配合 4bit 量化）
训练速度	较慢	较快
推理与部署	每个任务存一份完整模型	可合并，也可一个底座挂多个插件
训练风险	更容易过拟合，也更容易遗忘旧能力	低秩约束限制自由度，底座权重保持不动
适用场景	算力充足、确实需要全模型一起动	大多数业务微调的默认起点

关于学习率，多说一句。LoRA 的可训练参数量比全量微调少了两个数量级，如果沿用全量微调的学习率（1e−5 到 2e−5），更新幅度太小，LoRA 可能学不动，训练很多步 loss 还是不怎么下降。LoRA 的学习率通常需要大一个数量级，1e−4 到 3e−4 是比较常见的范围。如果你发现 LoRA 训练的 loss 下降特别慢，第一个该检查的就是学习率。

关于过拟合，LoRA 天然具有正则化效果。前面讲过拟合的时候说过，过拟合的核心原因是模型的自由度太大而训练数据太少。LoRA 的低秩约束大幅限制了模型能调整的自由度——你只给了它十几个方向的调整空间，它想“背答案”都没有足够的参数来记住每一条训练数据。

关于灾难性遗忘，这一点尤其值得注意。前面讲指令微调的时候提到过，微调有一个风险：模型在学新东西的过程中可能把预训练学到的旧知识覆盖掉。LoRA 天然缓解了这个问题：原始权重 W 完全冻住，预训练的记忆完好无损，LoRA 旁路只是在上面加了一层薄薄的修正。

面试考点 · 为什么 LoRA 的学习率要比全量微调大

常见问法：LoRA 微调时学习率通常设 1e−4 到 3e−4，全量微调是 1e−5 到 2e−5，为什么 LoRA 要大一个数量级？

典型错误：只回答“因为参数少所以要大”，说不清因果关系。

正确思路：“参数少 $\rightarrow$ 学习率要大”这个判断是对的，但还要理解为什么这个因果关系成立。深层原因是：LoRA 只有几百万个参数在更新，这些参数承担了“让模型行为发生变化”的全部责任。如果每一步更新的幅度和全量微调一样小，几百万参数挤出来的变化量就太小了——模型几乎感受不到这些修正。你需要让每个 LoRA 参数“走得更远一些”来补偿参数数量上的不足。另一方面，LoRA 参数的低秩约束本身就限制了它“走偏”的空间，所以用更大的学习率也不容易发散。这两个因素合在一起，决定了 LoRA 需要更大的学习率。

15.1.12 常见误区

到这里 LoRA 的核心思想已经讲完了，但有几个常见的认知误区需要专门纠正。

- **“LoRA 是退而求其次的方案，效果肯定不如全量微调。”**
  这是最普遍的误解。很多人觉得 LoRA 是“穷人版微调”——显存不够才不得不用，效果打了折扣。事实上，在大模型微调的绝大多数场景下，LoRA 和全量微调的效果差距非常小，有时 LoRA 甚至更好。**在工业界，LoRA 不是退路，而是默认选择。**只有在极少数对效果要求极其苛刻、同时算力充裕的场景下，才会考虑全量微调。
- **“r 越大效果越好。”**
  直觉上觉得 r 越大，LoRA 的表达能力越强。但实际上 r 从 8 增大到 64，在大多数任务上效果提升非常有限。r 继续往上加，反而可能因为可训练参数增多而过拟合。$r = 8$ 或 $r = 16$ 是一个非常稳妥的起步值。
  还有一个常见误会：显存紧张的时候，很多人第一反应是把 r 压小。但 r 从 64 降到 8，省下来的通常只是几百 MB；显存的大头是冻住的底座模型和中间激活值，跟 r 关系不大。真要省显存，优先考虑量化底座模型、减小 batch size，或者缩短序列长度。
- **“LoRA 省的是推理显存。”**
  LoRA 省的是训练显存。合并权重之后，模型和全量微调出来的完全一样大，推理时没有任何显存或速度的优势。如果不合并，推理时反而多了一点计算（走两条路再相加）。推理阶段的优化，是量化和 KV Cache 等后面会讲到的技术的事情。
- **“B 和 A 的初始化方式无所谓。”**

前面手算已经展示过了。B 必须初始化为零，A 随机初始化。这个不对称是刻意的设计，保证训练起点就是原始预训练模型。如果两个都随机初始化，训练开始时 LoRA 旁路会输出随机噪声，直接破坏预训练模型已有的能力。

15.1.13 LoRA 超参数一览

最后把 LoRA 涉及的几个超参数汇总在一起，方便你在实战的时候快速查阅：

超参数	含义	推荐起步值	注意事项
r（秩）	LoRA 旁路矩阵的秩，决定可训练参数量	16	不是越大越好，8 到 32 通常够用
α（缩放因子）	控制旁路的贡献比例，实际起作用的是 α/r	16（与 r 相等）	$\alpha = r$ 是最常见的做法
`target_modules`	LoRA 加在哪些权重矩阵上	`all-linear`	写错模块名不报错，一定要检查参数量
`lora_dropout`	旁路中的 Dropout 比例	0.05	数据量少时可适当调大到 0.1
学习率	虽不是 LoRA 独有，但在 LoRA 场景下范围不同	2e−4	比全量微调大约一个数量级

如果你是第一次用 LoRA，$r = 16$、$\alpha = 16$、`target_modules`设成`all-linear`、`lora_dropout`设成 0.05、学习率设成 2e−4。这套配置在大多数场景下是一个靠谱的起点，可以直接拿去跑，后续根据实验结果再微调。

LoRA 不是唯一的参数高效微调方法，但截至写作时，它是工业界最成熟、生态支持最好、使用最广泛的方案。下一节我们会用 PEFT 库把 LoRA 微调的完整流程跑一遍。

15.1.14 自检清单

1. LoRA 为什么能用远少于原始参数数量的参数来做微调？
   因为微调前后权重矩阵的变化量 ΔW 可以用低秩矩阵很好地近似，虽然维度很大，但有效信息集中在很少几个方向上。这个低秩近似可以被分解为两个小矩阵 B 和 A 的乘积，参数量大幅减少。
2. 为什么 B 初始化为零？
   为了保证训练起点就是原始预训练模型。B 全零意味着 LoRA 旁路在训练开始时输出为零，对模型没有任何影响。然后随着训练进行，旁路从零开始逐渐长出需要的修正量。
3. LoRA 省的是训练显存还是推理显存？
   训练显存。合并权重后，推理时模型和全量微调的完全一样大。
4. LoRA 微调的学习率应该比全量微调大还是小？
   大。LoRA 可训练参数少得多，学习率通常在 1e−4 到 3e−4，比全量微调的 1e−5 到 2e−5 大约一个数量级。
5. LoRA 里的 r 和 α 分别控制什么？为什么第一次不应该盲目把 r 设得很大？
   r 控制低秩旁路的容量，越大可训练参数越多，能表达的改动越复杂；α 控制 LoRA 更新量的缩放强度。第一次通常可以从 $r = 16$、$\alpha = 16$ 这类保守配置起步。盲目

增大 r 不一定提升效果，数据量不够时反而更容易过拟合。

15.1.15 Python 交互实验室

正文里用手算带你走过了 LoRA 的核心计算，但有几件事适合用图和数字继续拆开：秩为 1 的矩阵和满秩矩阵在热力图中的视觉差异、低秩近似的误差在 r 达到真实秩的瞬间突然归零的那个拐点，以及 B 随机初始化时模型输出被污染的程度。资源包里的`lora_core_demo.py`用四个小实验把这些现象一一呈现。

建议你先完整跑一遍看默认结果，然后回到代码开头的参数配置区，重点试两件事：一是在实验 2 中把`TRUE_RANK`从 4 改成 16，观察误差曲线的拐点如何跟着移动；二是在实验 3 中把`LORA_R`从 2 改成 1，看看旁路输出的每个维度是如何被压缩到只剩一个自由度的。

15.2 LoRA 实战：用最精简的代码跑通微调任务

15.2.1 你以为要从头学一套新东西

前面用了一整节的篇幅把 LoRA 的数学掰开揉碎讲了一遍：低秩分解、B 矩阵零初始化、缩放因子 α/r、可训练参数量从几十亿压到几百万。你大概已经对 LoRA 的原理有了不错的直觉。但“懂原理”和“能上手”之间，还差一步：代码到底怎么写？

如果你在跑通全量微调之前就看到“LoRA 实战”这个标题，心里多半会有点紧张，又是一套新流程要学。但你现在的处境完全不同。你已经亲手跑通过一次完整的指令微调了，从数据准备到训练到验证，每一步都走过一遍。那个 pipeline 里的每个环节，你都知道它在干什么。

好消息是：从全量微调切换到 LoRA 微调，代码层面的改动比你想象的少得多。数据处理一行不用改，Trainer 的用法也完全一样。如果只看训练启动前，真正需要改的地方主要有两处：第一，在模型加载之后、训练开始之前，多一步“给模型套上 LoRA”的操作；第二，把学习率调大一个数量级。等训练结束后，保存和加载还会多出 adapter 这套逻辑，后面我们单独讲。

如果你把全量微调的代码和 LoRA 微调的代码贴在一起做一次文件对比，差异大概只有十来行。其余几十行代码一模一样。

LoRA 的设计目标就是尽量不打扰已有的训练流程。它只管冻结参数、挂上旁路，剩下的事情，比如前向传播、算 loss、反向传播、更新参数，全都照旧。所以你已经会的那套代码，几乎原样搬过来就能用。

这一节的目标就是带你亲眼验证这一点。我们会用同一份数据、同一套数据处理流程，把全量微调换成 LoRA 微调，然后看看效果有什么不同。

在正式写代码之前，有一件事值得先说清楚。在实际工作中，大多数团队做 LoRA 微调用的并不是我们这种一步步手写的方式。业界有不少封装好的微调框架，比如 LLaMA-Factory、Swift 等，配一个 yaml 文件、敲一行命令就能启动训练，整个过程可能五分钟就搞定了。如果你入职之后发现同事都在用这类框架，不要觉得本节的内容“落伍了”。恰恰相反，框架帮你做的每一件事，你都需要知道它在干什么。等你对整个流程了然于胸了，再去用框架，你会很清楚每个配置项对应的是哪一步操作，出了问题也知道往哪里查。而且框架迭代很快，今年流行的框架明年可能就换了另一个，但底层的逻辑不会变。